MATLAB Review

1: Arrays, Indexing, and Linear Systems

Personal study notes

Purpose: Revisit the matrix operations needed for Lecture 2.

Contents

1 Rows, columns, and dimensions 1

2 Indexing and selections 1

3 Matrix operations versus element-wise operations 2

4 Solve a small system and check the model 2

5 Function handles, a loop, and a plot 2

6 Run the checks and practise 3

7 Sources and editorial remarks 3

1 Rows, columns, and dimensions

A decision vector is usually a column. Commas or spaces separate entries within a row; semicolons separate rows. A semicolon after a statement suppresses its output.

```
x = [2; 6];                 % 2-by-1 column
c = [3; 2];                 % 2-by-1 column
M = [1, 2, 3; 4, 5, 6];     % 2-by-3 matrix
size(M)                     % Expected: [2 3]
numel(M)                    % Expected: 6
revenue = c.' * x;          % Expected: 18
```

The revenue product has dimensions $(1 \times 2)(2 \times 1) = 1 \times 1$. `.'` transposes without conjugating; `'` also conjugates complex entries. They agree for real data. Prefer `size` for matrix dimensions: `length` gives the largest dimension and does not count all elements.

2 Indexing and selections

Indices start at 1. The first index selects the row; the second selects the column. A colon selects all entries along that dimension. Linear indexing reads columns from top to bottom, then proceeds to the next column.

```
M(2, 3)                     % 6
M(:, 2)                     % [2; 5]
M(1, :)                     % [1 2 3]
M(:)                        % [1; 4; 2; 5; 3; 6]
M(end, :)                   % Last row: [4 5 6]
```

```
M(M > 3)                    % [4; 5; 6]
```

Logical indexing selects entries satisfying a condition. For a collection of candidate decision vectors stored as columns, select whole columns with a logical mask rather than extracting individual entries.

3 Matrix operations versus element-wise operations

```
M = [1, 2; 3, 4];
M * M                       % [7 10; 15 22]
M .* M                      % [1 4; 9 16]
M ^ 2                       % Same as M*M
M .^ 2                      % Same as M.*M
```

For matrix multiplication, $(M^2)_{ij} = \sum_k M_{ik}M_{kj}$; for element-wise squaring, $(M \wedge 2)_{ij} = M_{ij}^2$. The dot changes the mathematical operation. For sampled function values, typically use `.*`, `./`, and `.^`; for a linear model Ax, use matrix multiplication.

Remark 3.1 (Implicit expansion can hide shape mistakes)**.** Adding a 2×1 column to a 1×2 row produces a 2×2 array by implicit expansion. It need not raise a dimension error. Inspect dimensions when an objective or feasibility calculation unexpectedly returns a matrix.

4 Solve a small system and check the model

The first two tight production constraints from Lecture 2 give

$$2x_1 + x_2 = 10, \qquad 3x_1 + 3x_2 = 24.$$

Dividing the second equation by 3 gives $x_1 + x_2 = 8$. Subtraction yields $x_1 = 2$, then $x_2 = 6$. MATLAB expresses this system as:

```
B = [2, 1; 3, 3];
b = [10; 24];
x = B \ b;                  % Expected: [2; 6]
residual = norm(B*x - b, Inf);
assert(residual <= 1e-10);
```

Backslash solves the linear system. For this nonsingular square B, the solution is unique; avoid calculating `inv(B)*b`. For general matrices, inspect rank, conditioning, warnings, and residuals: a small residual alone does not prove a small error in the solution.

Check *all* constraints of the original model as well:

```
A = [2, 1; 3, 3; 1.5, 0];
capacity = [10; 24; 6];
slack = capacity - A*x;     % Expected: [0; 0; 3]
assert(all(x >= -1e-10));
assert(all(slack >= -1e-10));
```

These verify feasibility. They do not search for an LP optimum; Lecture 2's hand certificate proves the objective cannot exceed 18 PLN. For these small examples 10^{-10} is an absolute tolerance. Real problems require tolerances chosen with their units, scaling, and numerical conditioning.

5 Function handles, a loop, and a plot

Consider the quadratic $f(t) = t^2 - 4t + 5 = (t-2)^2 + 1$. Its global minimum is $f(2) = 1$.

```
f = @(t) t.^2 - 4*t + 5;
t = linspace(0, 4, 101);
values = f(t);
loopValues = zeros(size(t));
for k = 1:numel(t)
    loopValues(k) = f(t(k));
end
assert(max(abs(values-loopValues)) <= 1e-10);
```

The function handle can be called on a scalar or the entire sample vector because the square is element-wise. Preallocate the loop output instead of repeatedly growing an array. Agreement between the loop and vectorized evaluation checks implementation consistency; the completing-the-square argument establishes the minimum.

```
fig = figure('Visible', 'off');
plot(t, values, 'LineWidth', 1.5);
xlabel('t'); ylabel('f(t)');
title('f(t) = (t - 2)^2 + 1'); grid on;
exportgraphics(fig, 'quadratic-review.png', 'Resolution', 150);
close(fig);
```

The accompanying script saves its figure to the `matlab/output/` directory using the script location to find the repository root. A sampled graph illustrates the function; it does not replace a proof or guarantee that a minimum lies on the sampled grid.

6 Run the checks and practise

From the repository root, run:

```
run('matlab/review/arrays_and_linear_systems.m')
```

The script contains assertions for the hand results above. Before running, predict the results, then explain any difference.

1. Write a column vector with entries 1 through 5 and select its last two entries.
2. Compute both the matrix square and element-wise square of a 2×2 matrix.
3. Replace the production candidate by $(4, 4)^T$. Which resource constraint fails?
4. Change the quadratic to $t^2 - 6t + 10$. Derive its minimum by hand before plotting.

7 Sources and editorial remarks

- MathWorks: Matrices and Arrays: array construction, dimensions, and transpose.
- MathWorks: Array Indexing: subscripts, linear indices, and logical masks.
- MathWorks: Array vs. Matrix Operations: operator distinctions and compatible array sizes.
- MathWorks: mldivide: linear system solution and numerical warnings.
- MathWorks: exportgraphics: graphics export.

Polish terms are maintained in `docs/glossary.md`: *wektor wierszowy*, *wektor kolumnowy*, *indeksowanie*, *operacje elementowe*, *uchwyt do funkcji*, and *reszta układu równań*.

Provenance and verification. **Sources:** Official MathWorks documentation listed at the end; Lecture 1's historical refresher scope motivates this topic.
Execution status: Verified on 2026-10-04 in MATLAB 26.2.0.3386108 (R2026b), based on output supplied by the user from a normal terminal. All numerical assertions passed and the plot was exported. Hand results agree with the script assertions.

The practice prompts in Section 6 are original examples. Full terminal output is recorded in `docs/matlab-verification.md`.